

Modern Processor Design (III): Whenever You're Ready

Hung-Wei Tseng

Demo revisited: evaluating the cost of mis-predicted branches

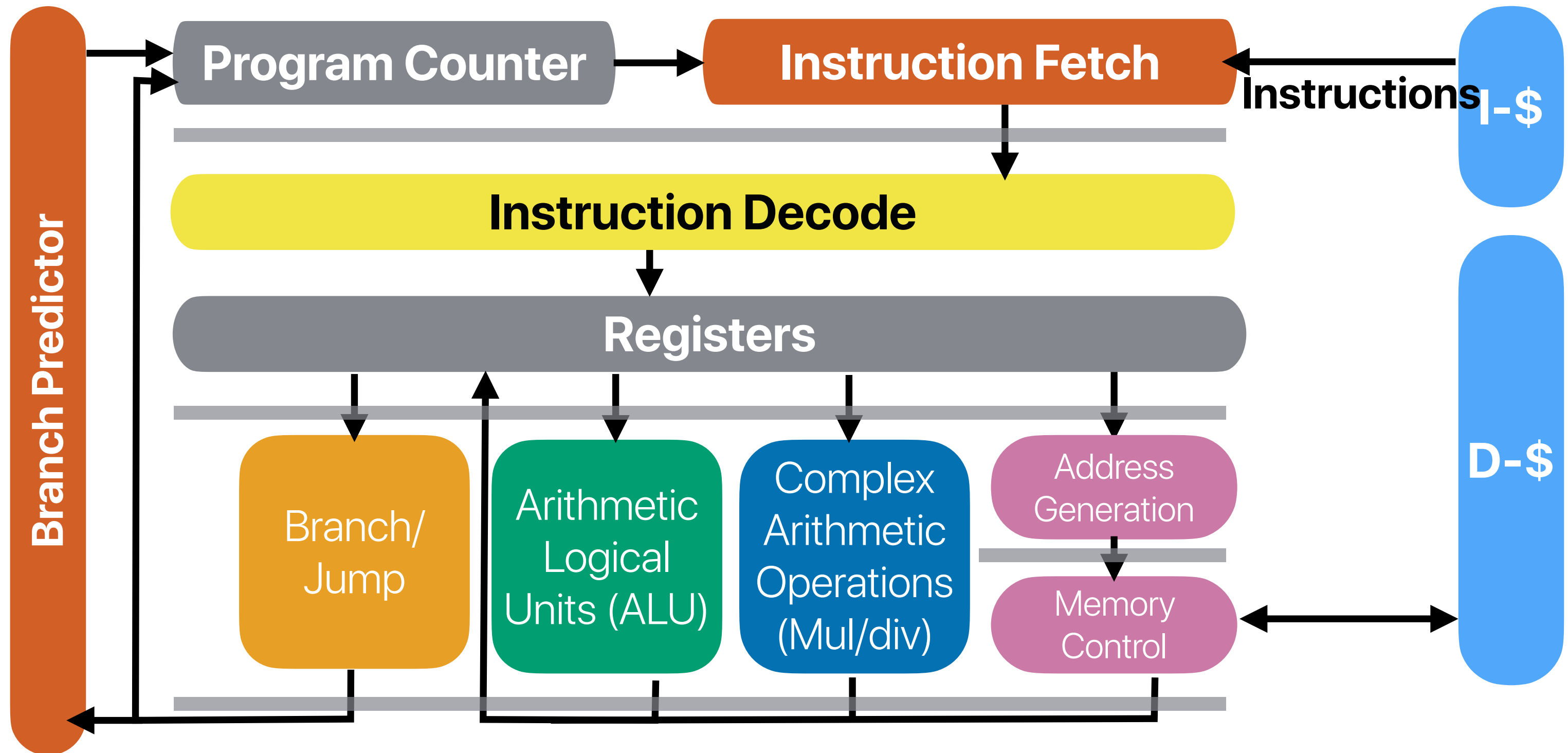
- Compare the number of mis-predictions
- Calculate the difference of cycles
- We can get the "average CPI" of a mis-prediction!

34 cycles on Intel Alder Lake

23 cycles on AMD Zen 3

Could be more expensive than cache misses

Recap: Microprocessor with a "branch predictor"



Takeaways: branch predictions

- The cost of not to predict a branch is to stall until the data dependency is resolved — 34 cycles on modern intel processors and 23 on AMD processors
- Branch predictions allow the processor to at least make some progress and hide the stalls if we guessed correctly!
- Dynamic branch prediction — predict based on prior history
 - Local predictor — make predictions based on the state of each branch instruction
 - Global predictor — make predictions based on the state from all branches
 - Both are not perfect — hybrid predictors
 - Tournament
 - Perceptron
- All modern processors have pretty accurate branch predictors — if the code itself is predictable

Recap: But A is faster!

using one statement

d. /* one line statement using bit-wise operators */ (most efficient)
`a^=b^=a^=b;`

The order of evaluation is from right to left. This is same as in approach (c) but the three statements are compounded into one statement.

A

```
void regswap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

B

```
void xorswap(int* a, int* b) {  
    *a ^= *b = *a = *b;  
}
```

Data hazards

Data hazards

- An instruction currently in the pipeline cannot receive the “logically” correct value for execution
- Data dependencies
 - The output of an instruction is the input of a later instruction
 - **May sometimes** result in data hazard if the later instruction that consumes the result is still in the pipeline



How many data dependencies do we have?

- How many pairs of data dependencies are there in the following x86 instructions?

```
movl    (%rdi), %eax
movl    (%rsi), %edx
movl    %edx, (%rdi)
movl    %eax, (%rsi)
```

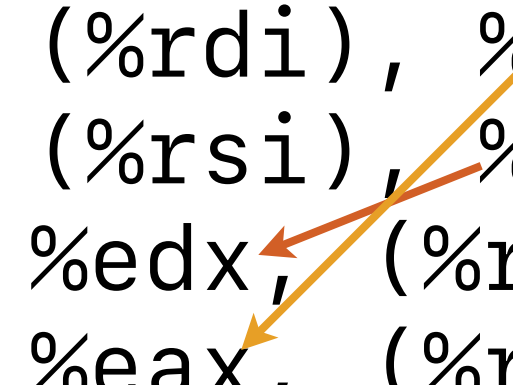
```
int temp = *a;
*a = *b;
*b = temp;
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5

How many dependencies do we have?

- How many pairs of data dependences are there in the following x86 instructions?

```
movl    (%rdi), %eax
movl    (%rsi), %edx
movl    %edx, (%rdi)
movl    %eax, (%rsi)
```



```
int temp = *a;
*a = *b;
*b = temp;
```

A. 1

B. 2

C. 3

D. 4

E. 5



How many data dependencies do we have?

- How many pairs of data dependencies are there in the following x86 instructions?

```
movl    (%rdi), %eax
xorl    (%rsi), %eax
movl    %eax, (%rdi)
xorl    (%rsi), %eax
movl    %eax, (%rsi)
xorl    %eax, (%rdi)
```

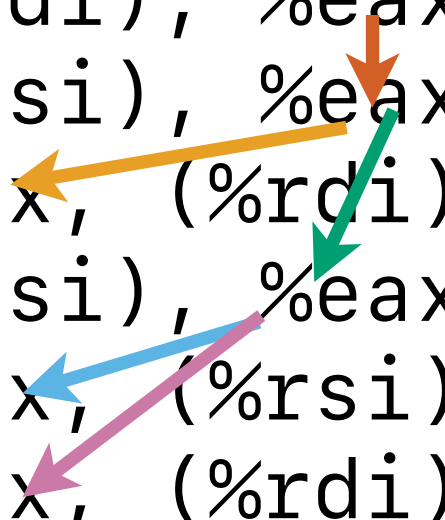
```
*a ^= *b;
*b ^= *a;
*a ^= *b;
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5

How many dependencies do we have?

- How many pairs of data dependencies are there in the following x86 instructions?

```
movl    (%rdi), %eax
xorl    (%rsi), %eax
movl    %eax, (%rdi)
xorl    (%rsi), %eax
movl    %eax, (%rsi)
xorl    %eax, (%rdi)
```



```
*a ^= *b;
*b ^= *a;
*a ^= *b;
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5

Data hazards

① movl (%rdi), %eax

② movl (%rsi), %edx

③ movl %edx, (%rdi)

④ movl %eax, (%rsi)

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5		(4)	(3)	(2)	(1)			
6			(4)	(3)	(2)	(1)		
7				(4)	(3)	(2)	(1)	
8					(4)	(3)	(2)	
9						(4)	(3)	
10							(4)	
11								
12								
13								
14								

%edx does not have our desired value

%eax does not have our desired value

Solution 1: Let's try "stall" again

- Whenever the input is not ready when the consumer is decoding, just stall — the consumer stays at ID.



Data hazards?

- How many cycles do we have to stall in the following x86 instructions to get the expected output if a memory operation (assume 100% cache hit rate) takes 5 cycles?

① `movl (%rdi), %eax`
② `movl (%rsi), %edx`
③ `movl %edx, (%rdi)`
④ `movl %eax, (%rsi)`

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5



Data hazards?

- How many cycles do we have to stall in the following x86 instructions to get the expected output if a memory operation (assume 100% cache hit rate) takes 5 cycles?

```
① movl    (%rdi), %eax
```

```
② movl    (%rsi), %edx
```

```
③ movl    %edx, (%rdi)
```

```
④ movl    %eax, (%rsi)
```

- A. 1

- ## B. 2

- C. 3

- ## D. 4

- ## E. 5

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(4)	(3)		(2)	(1)			
6	(4)	(3)			(2)	(1)		
7	(4)	(3)				(2)	(1)	
8	(4)	(3)					(2)	(1)
9	(4)	(3)						(2)
10		(4)	(3)					
11			(4)	(3)				
12				(4)	(3)			
13					(4)	(3)		
14						(4)		(3)
15								(4)

we have the value for %edx already!

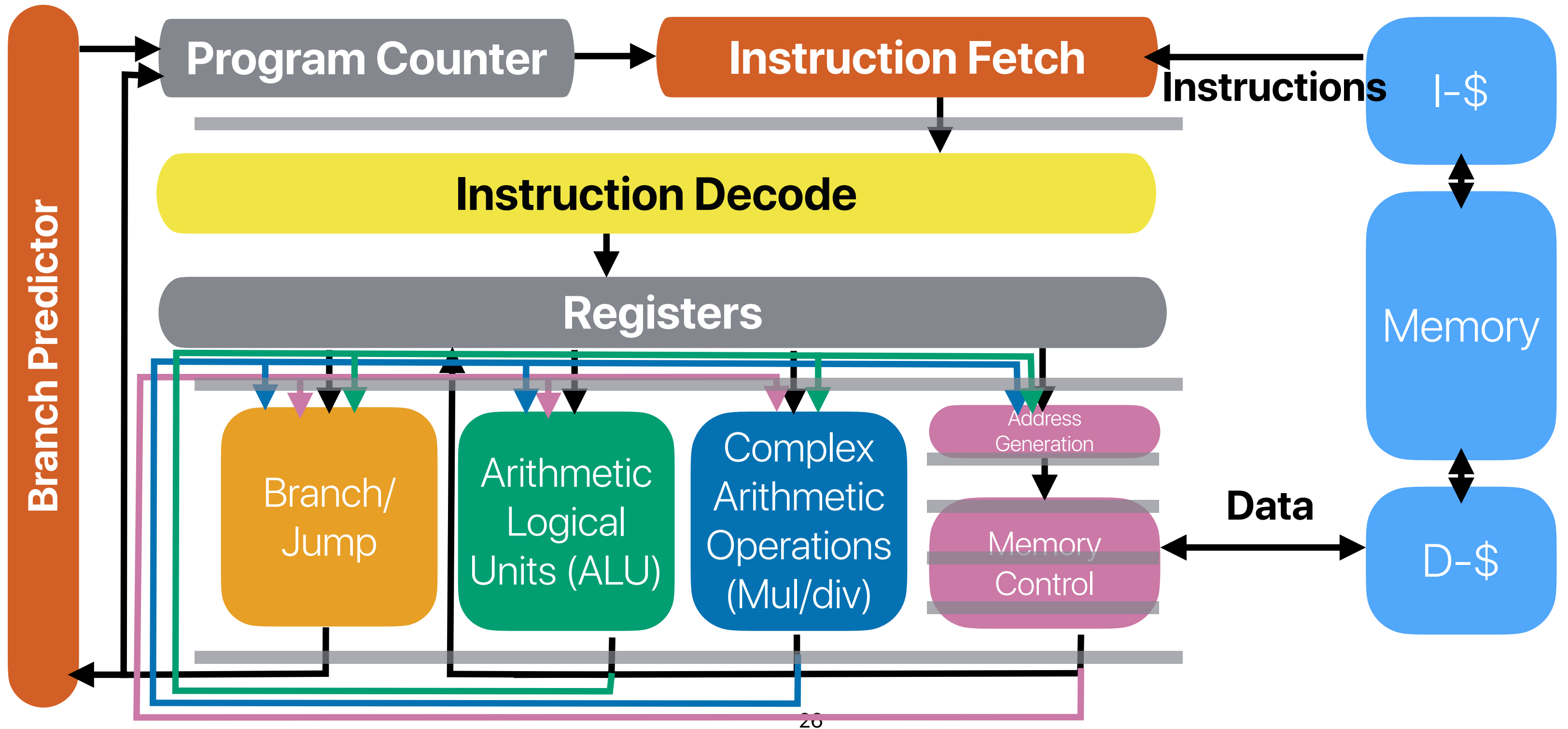
Why another cycle?

5 cycles stalls

Solution 2: Data forwarding

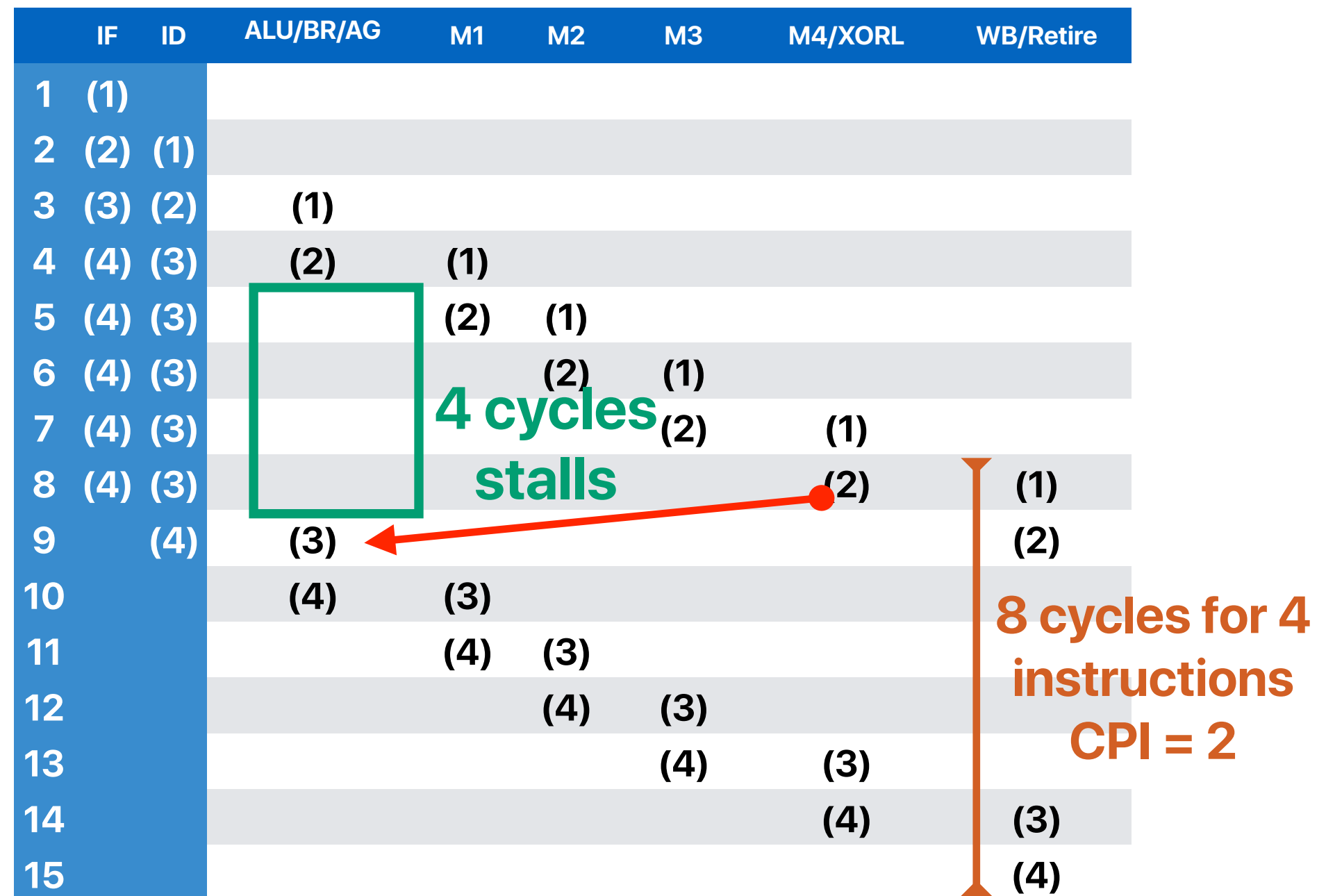
- Add logics/wires to forward the desired values to the demanding instructions

Data "forwarding"



The effect of data forwarding

- ① `movl (%rdi), %eax`
- ② `movl (%rsi), %edx`
- ③ `movl %edx, (%rdi)`
- ④ `movl %eax, (%rsi)`





How many of data hazards w/ Data Forwarding?

- How many pairs of back-to-back data dependences in the following x86 instructions will result in stalls even with data forwarding and both memory operations & xorl take 5 cycles?

① movl (%rdi), %eax
② xorl (%rsi), %eax
③ movl %eax, (%rdi)
④ xorl (%rsi), %eax
⑤ movl %eax, (%rsi)
⑥ xorl %eax, (%rdi)

- A. 0
B. 1
C. 2
D. 3
E. 4

*a ^= *b;
*b ^= *a;
*a ^= *b;



How many of data hazards w/ Data Forwarding?

- How many pairs of back-to-back data dependences in the following x86 instructions will result in stalls even with data forwarding and both memory operations & xorl take 5 cycles?

① movl (%rdi), %eax
② xorl (%rsi), %eax
③ movl %eax, (%rdi)
④ xorl (%rsi), %eax
⑤ movl %eax, (%rsi)
⑥ xorl %eax, (%rdi)

A. 0

B. 1

C. 2

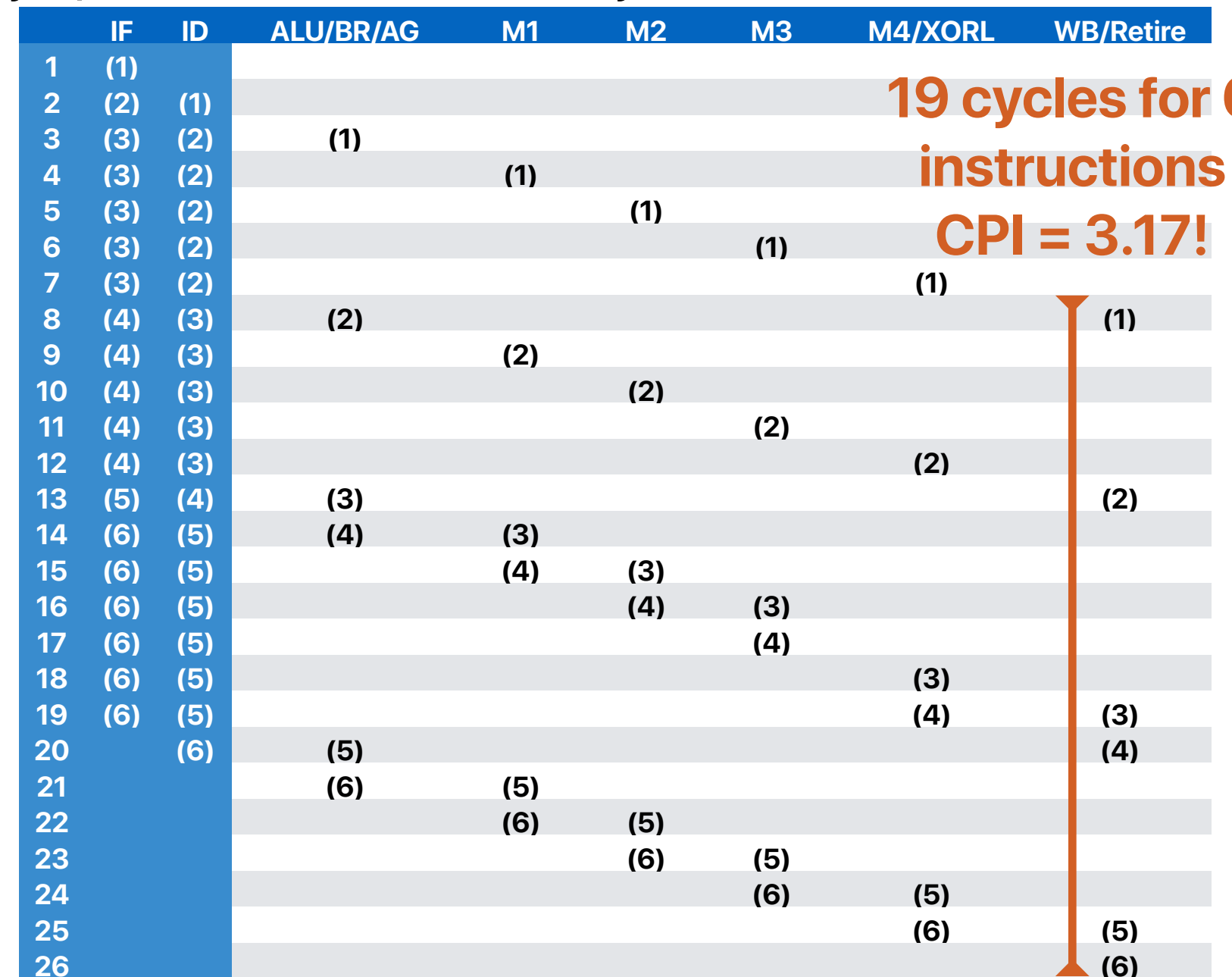
D. 3

E. 4

*a ^= *b;

*b ^= *a;

*a ^= *b;



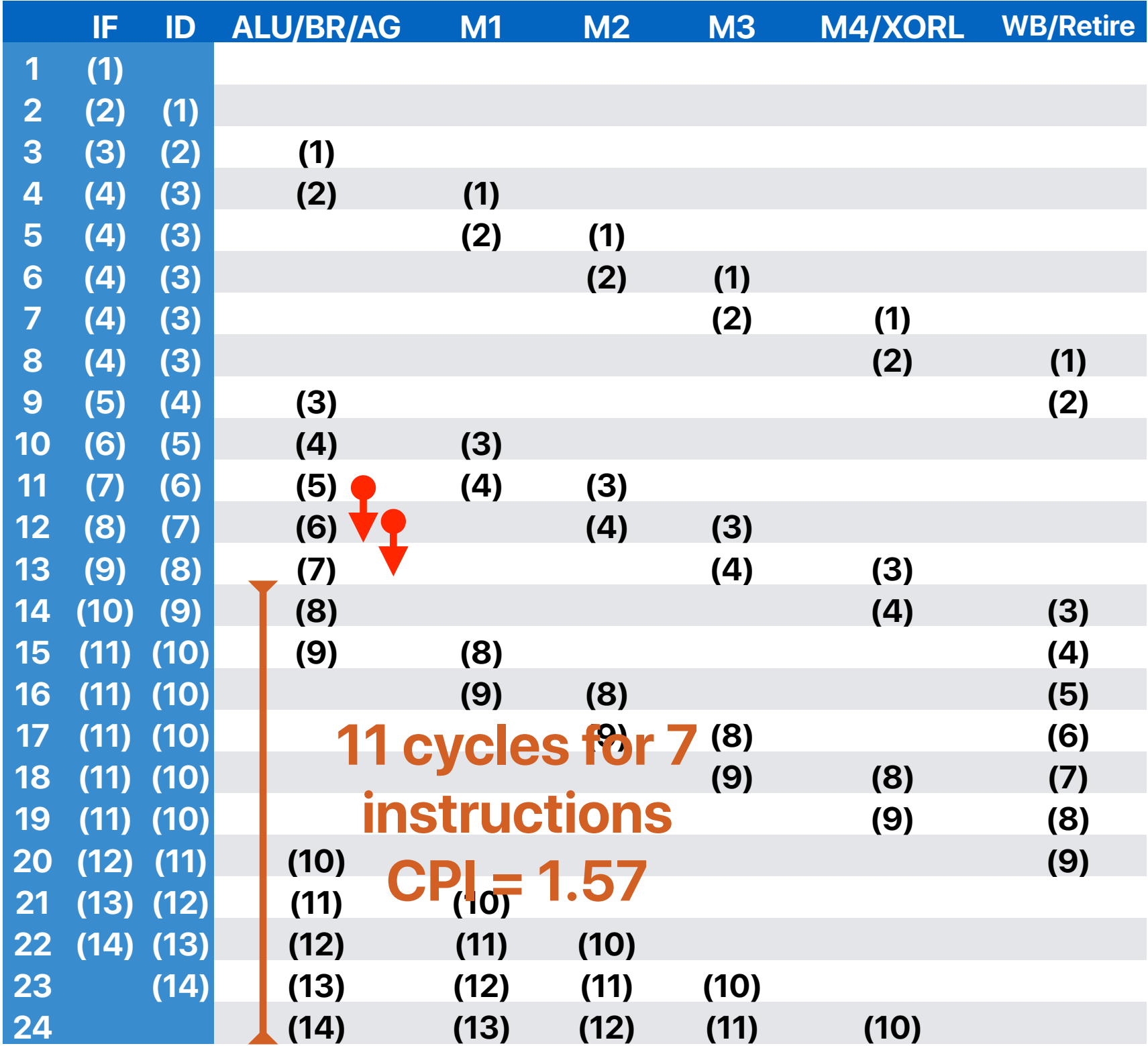
Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient

Let's extend the example a bit...

```
for(i = 0; i < count; i++) {  
    int64_t temp = a[i];  
    a[i] = b[i];  
    b[i] = temp;  
}
```

```
.L9:  
① movq    (%rdi,%rax), %rsi  
② movq    (%rcx,%rax), %r8  
③ movq    %r8, (%rdi,%rax)  
④ movq    %rsi, (%rcx,%rax)  
⑤ addq    $8, %rax  
⑥ cmpq    %r9, %rax  
⑦ jne     .L9  
⑧ movq    (%rdi,%rax), %rsi  
⑨ movq    (%rcx,%rax), %r8  
⑩ movq    %r8, (%rdi,%rax)  
⑪ movq    %rsi, (%rcx,%rax)  
⑫ addq    $8, %rax  
⑬ cmpq    %r9, %rax  
⑭ jne     .L9
```





The effect of code optimization

- By reordering which pair of the following instruction stream can we reduce stalls without affecting the correctness of the code?

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
```

- A. (1) & (2)
- B. (2) & (3)
- C. (3) & (5)
- D. (4) & (6)
- E. No ordering can help reduce the stalls



The effect of code optimization

- By reordering which pair of the following instruction stream can we reduce stalls without affecting the correctness of the code?

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
```

A. (1) & (2)

B. (2) & (3)

C. (3) & (5)

D. (4) & (6)

E. No ordering can help reduce the stalls

Compiler optimization

```
for(i = 0; i < count; i++) {  
    int64_t temp = a[i];  
    a[i] = b[i];  
    b[i] = temp;  
}
```

```
movq    (%rdi,%rax), %rsi  
movq    (%rcx,%rax), %r8  
movq    %r8, (%rdi,%rax)  
movq    %rsi, (%rcx,%rax)  
addq    $8, %rax  
cmpq    %r9, %rax  
jne     .L9  
movq    (%rdi,%rax), %rsi  
movq    (%rcx,%rax), %r8  
movq    %r8, (%rdi,%rax)  
movq    %rsi, (%rcx,%rax)  
addq    $8, %rax  
cmpq    %r9, %rax  
jne     .L9
```



.L9:

```
① movq    (%rcx,%rax), %r8  
② movq    (%rdi,%rax), %rsi  
③ addq    $8, %rax  
④ movq    %r8, -8(%rdi,%rax)  
⑤ movq    %rsi, -8(%rcx,%rax)  
⑥ cmpq    %r9, %rax  
⑦ jne     .L9  
⑧ movq    (%rcx,%rax), %r8  
⑨ movq    (%rdi,%rax), %rsi  
⑩ addq    $8, %rax  
⑪ movq    %r8, -8(%rdi,%rax)  
⑫ movq    %rsi, -8(%rcx,%rax)  
⑬ cmpq    %r9, %rax  
⑭ jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(5)	(4)			(2)	(1)		
7	(5)	(4)				(2)	(1)	
8	(6)	(5)	(4)				(2)	(1)
9	(7)	(6)	(5)	(4)				(2)
10	(8)	(7)	(6)	(5)	(4)			(3)
11	(9)	(8)	(7)		(5)	(4)		
12	(10)	(9)	(8)			(5)	(4)	
13	(11)	(10)	(9)	(8)			(5)	(4)
14	(11)	(10)	(10)	(9)	(8)			(5)
15	(11)	(10)			(9)	(8)		(6)
16	(11)	(10)				(9)	(8)	(7)
17	(12)	(11)	(11)				(9)	(8)
18	(12)	(11)	(12)	(11)				(9)
19	(13)	(12)	(13)	(12)	(11)			(10)
20	(14)	(13)	(14)		(12)	(11)		
21		(14)				(12)	(11)	
22							(12)	(11)
23								(12)
24								(13)
25								(14)

9 cycles for 7 instructions
CPI = 1.29



Missing opportunities

```
for(i = 0; i < count; i++) {  
    int64_t temp = a[i];  
    a[i] = b[i];  
    b[i] = temp;  
}
```

Compiler can only do this when it's 100% for sure count is always an even number! — loop unrolling

Compilers are limited by the number of registers available to the software!

```
:  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9
```

```
.L9:  
① movq    (%rcx,%rax), %r8  
② movq    (%rdi,%rax), %rsi  
③ addq    $8, %rax  
④ movq    %r8, -8(%rdi,%rax)  
⑤ movq    %rsi, -8(%rcx,%rax)  
⑥ movq    (%rcx,%rax), %r8  
⑦ movq    (%rdi,%rax), %rsi  
⑧ cmpq    %r9, %rax  
⑨ jne     .L9  
⑩ addq    $8, %rax  
⑪ movq    %r8, -8(%rdi,%rax)  
⑫ movq    %rsi, -8(%rcx,%rax)  
⑬ cmpq    %r9, %rax  
⑭ jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)				
6	(6)	(5)	(4)	(3)	(2)	(1)		
7	(7)	(6)	(5)	(4)	(3)	(2)	(1)	
8	(8)	(7)	(6)	(5)	(4)	(3)	(2)	(1)
9	(9)	(8)	(7)	(6)	(5)	(4)	(3)	(2)
10	(10)	(9)	(8)	(7)	(6)	(5)	(4)	(3)
11	(11)	(10)	(9)	(8)	(7)	(6)	(5)	(4)
12	(12)	(11)	(10)	(9)	(8)	(7)	(6)	(5)
13	(13)	(12)	(11)	(10)	(9)	(8)	(7)	(6)
14	(14)	(13)	(12)	(11)	(10)	(9)	(8)	(7)
15	(15)	(14)	(13)	(12)	(11)	(10)	(9)	(8)
16	(16)	(15)	(14)	(13)	(12)	(11)	(10)	(9)
17	(17)	(16)	(15)	(14)	(13)	(12)	(11)	(10)
18	(18)	(17)	(16)	(15)	(14)	(13)	(12)	(11)
19	(19)	(18)	(17)	(16)	(15)	(14)	(13)	(12)
20	(20)	(19)	(18)	(17)	(16)	(15)	(14)	(13)
21	(21)	(20)	(19)	(18)	(17)	(16)	(15)	(14)
22	(22)	(21)	(20)	(19)	(18)	(17)	(16)	(15)
23	(23)	(22)	(21)	(20)	(19)	(18)	(17)	(16)

7 cycles for 7 instructions
CPI = 1

Limitations of Compiler Optimizations

- If the hardware (e.g., pipeline changes), the same compiler optimization may not be that helpful
- The compiler can only optimize on static instructions, but cannot optimize dynamic instruction
 - Compiler cannot predict branches
 - Compiler does not know if cache has the data/instructions

Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent

Missing opportunities

```
for(i = 0; i < count; i++) {  
    int64_t temp = a[i];  
    a[i] = b[i];  
    b[i] = temp;  
}
```

Processor can predict what
should happen and unroll the
loop "dynamically"

```
:  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9  
movq    (%rcx,%rax), %r8  
movq    (%rdi,%rax), %rsi  
addq    $8, %rax  
movq    %r8, -8(%rdi,%rax)  
movq    %rsi, -8(%rcx,%rax)  
cmpq    %r9, %rax  
jne     .L9  
:  
① movq    (%rcx,%rax), %r8  
② movq    (%rdi,%rax), %rsi  
③ addq    $8, %rax  
④ movq    %r8, -8(%rdi,%rax)  
⑤ movq    %rsi, -8(%rcx,%rax)  
⑥ movq    (%rcx,%rax), %r8  
⑦ movq    (%rdi,%rax), %rsi  
⑧ cmpq    %r9, %rax  
⑨ jne     .L9  
⑩ addq    $8, %rax  
⑪ movq    %r8, -8(%rdi,%rax)  
⑫ movq    %rsi, -8(%rcx,%rax)  
⑬ cmpq    %r9, %rax  
⑭ jne     .L9
```

	IF	ID	ALU/BR/AG	M1	M2	M3	M4/XORL	WB/Retire
1	(1)							
2	(2)	(1)						
3	(3)	(2)	(1)					
4	(4)	(3)	(2)	(1)				
5	(5)	(4)	(3)	(2)	(1)			
6	(6)	(5)	(4)	(3)	(2)	(1)		
7	(7)	(6)	(5)	(4)	(3)	(2)	(1)	
8	(8)	(7)	(6)	(5)	(4)	(3)	(2)	(1)
9	(9)	(8)	(7)	(6)	(5)	(4)	(3)	(2)
10	(10)	(9)	(8)	(7)	(6)	(5)	(4)	(3)
11	(11)	(10)	(9)	(8)	(7)	(6)	(5)	(4)
12	(12)	(11)	(10)	(9)	(8)	(7)	(6)	(5)
13	(13)	(12)	(11)	(10)	(9)	(8)	(7)	(6)
14	(14)	(13)	(12)	(11)	(10)	(9)	(8)	(7)
15	(15)	(14)	(13)	(12)	(11)	(10)	(9)	(8)
16	(16)	(15)	(14)	(13)	(12)	(11)	(10)	(9)
17	(17)	(16)	(15)	(14)	(13)	(12)	(11)	(10)
18	(18)	(17)	(16)	(15)	(14)	(13)	(12)	(11)
19	(19)	(18)	(17)	(16)	(15)	(14)	(13)	(12)
20	(20)	(19)	(18)	(17)	(16)	(15)	(14)	(13)
21	(21)	(20)	(19)	(18)	(17)	(16)	(15)	(14)
22	(22)	(21)	(20)	(19)	(18)	(17)	(16)	(15)
23	(23)	(22)	(21)	(20)	(19)	(18)	(17)	(16)

7 cycles for 7
instructions
CPI = 1

Dynamic instruction scheduling/ Out-of-order (OoO) execution

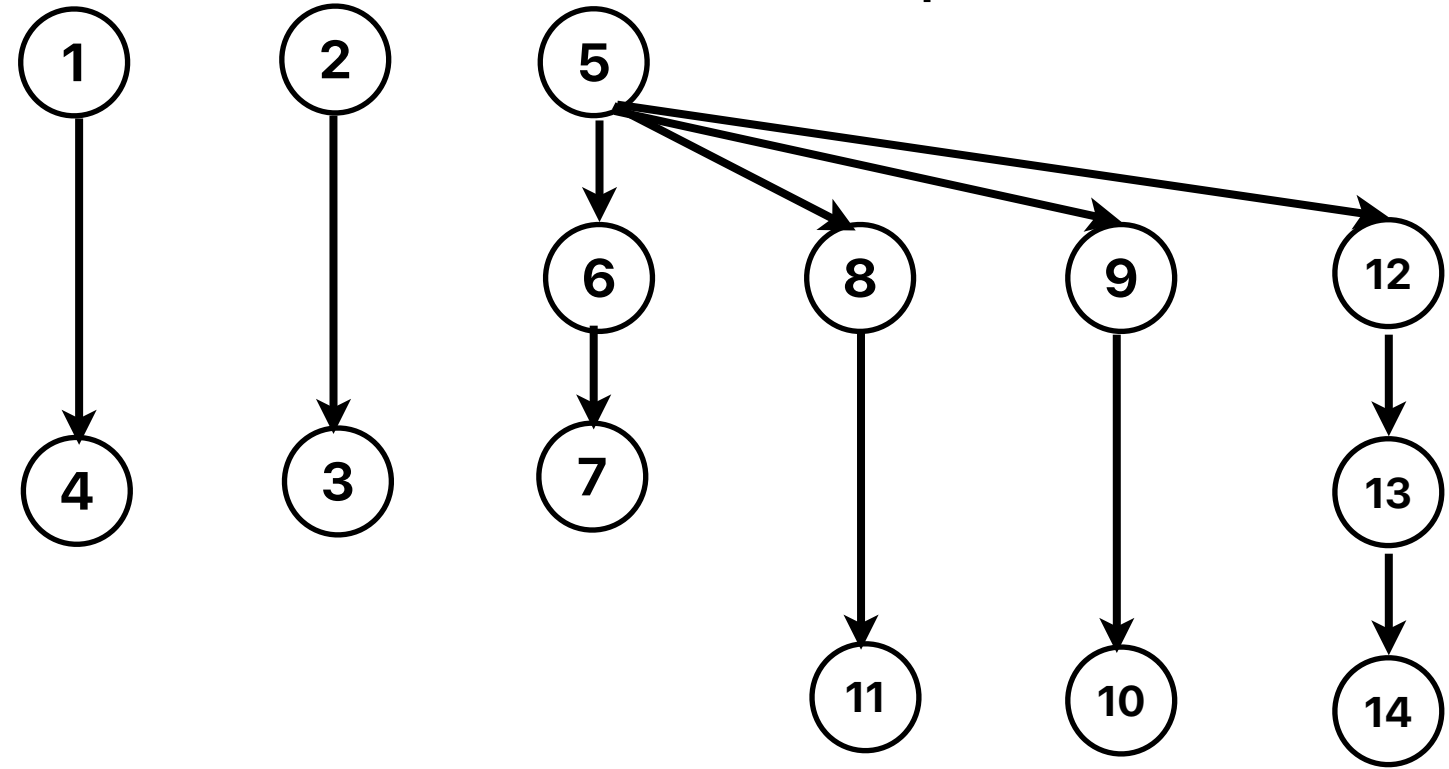
What do you need to execution an instruction?

- Whenever the instruction is decoded — put decoded instruction somewhere
- Whenever the inputs are ready — **all data dependencies are resolved**
- Whenever the target functional unit is available

Scheduling instructions: based on data dependencies

- Draw the data dependency graph, put an arrow if an instruction depends on the other.

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



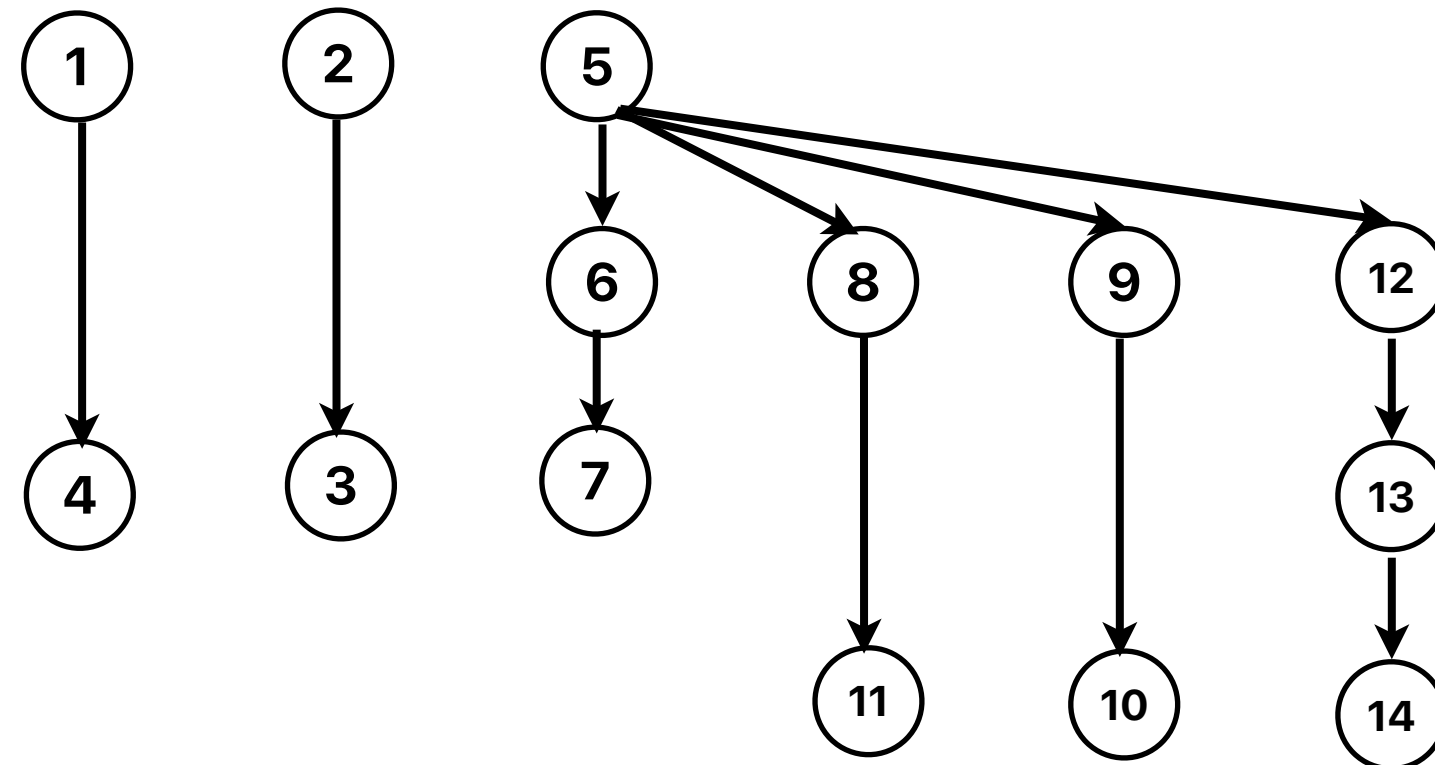
- **In theory**, instructions without dependencies can be executed in parallel or out-of-order
- Instructions with dependencies (on the same path) can never be reordered



If we can predict the future ...

- Consider the following dynamic instructions:

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



Which of the following pair can we reorder without affecting the correctness if the **branch prediction is perfect**?

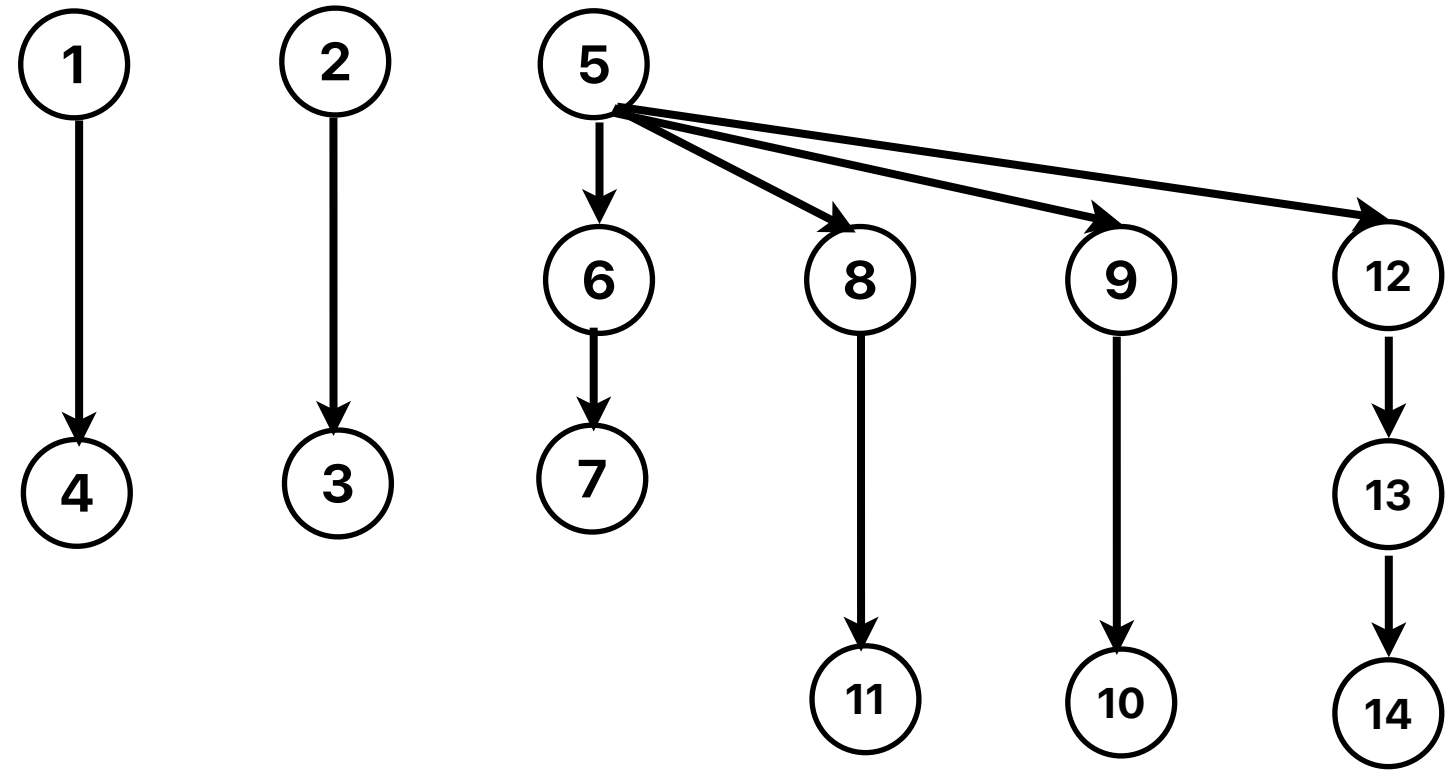
- A. (1) and (2)
- B. (3) and (5)
- C. (4) and (6)
- D. (6) and (8)
- E. (3) and (8)



If we can predict the future ...

- Consider the following dynamic instructions:

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



Which of the following pair can we reorder without affecting the correctness if the **branch prediction is perfect**?

A. (1) and (2)

B. (3) and (5)

C. (4) and (6)

D. (6) and (8)

E. (3) and (8)

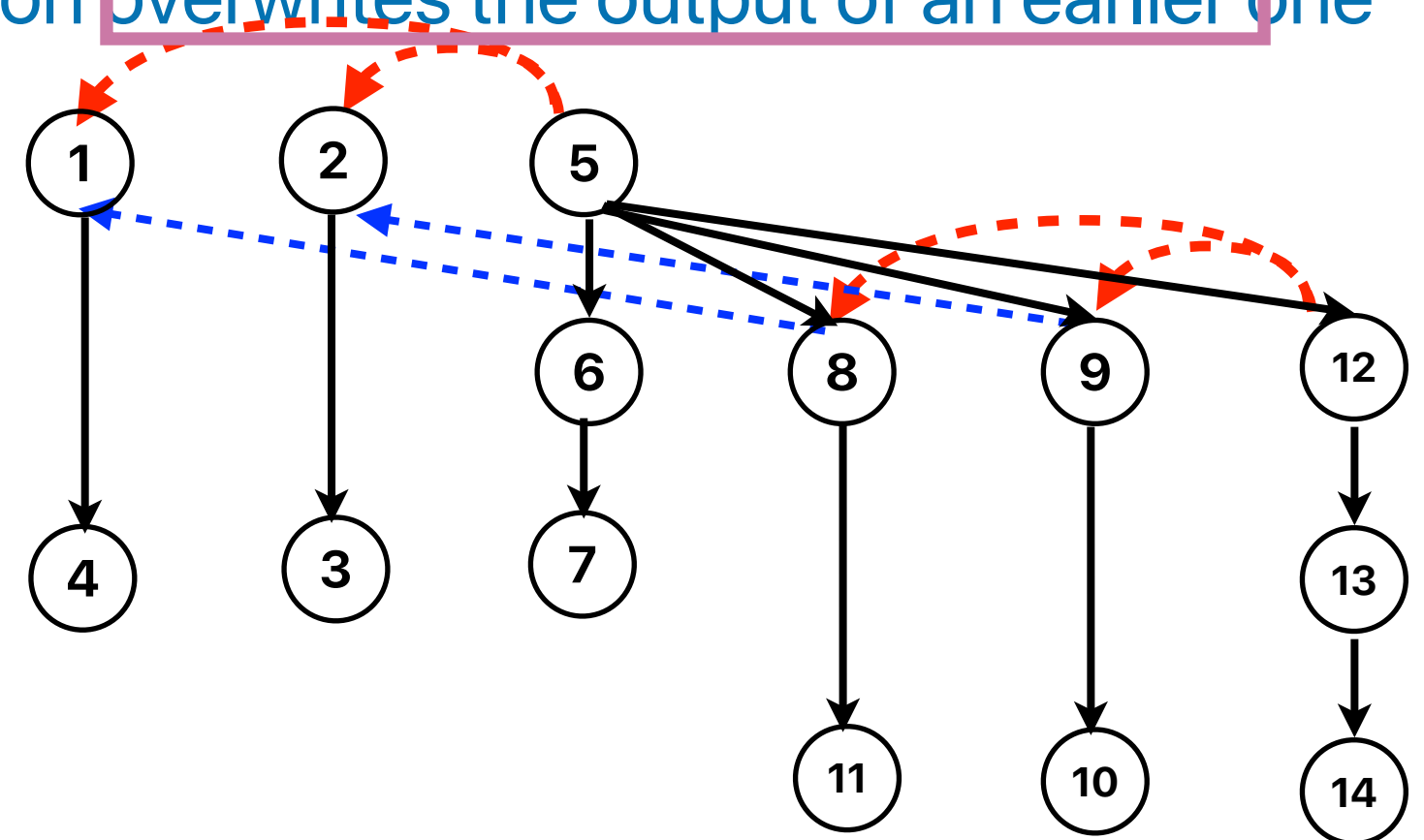
False dependencies

- We are still limited by **false dependencies**
- They are not “true” dependencies because they don’t have an arrow in data dependency graph
 - **WAR (Write After Read):** a later instruction overwrites the source of an earlier one
 - 5 and 1, 5 and 2, 12 and 8, 12 and 9
 - **WAW (Write After Write):** a later instruction overwrites the output of an earlier one

- 8 and 1
- 9 and 2

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```

54



False dependencies

- We are still limited by **false dependencies**
- They are not “true” dependencies because they don’t have an arrow in data dependency graph
 - WAR (Write After Read): a later instruction overwrites the source of an earlier one
 - 5 and 1, 5 and 2, 12 and 8, 12 and 9
 - WAW (Write After Write): a later instruction overwrites the output of an earlier one

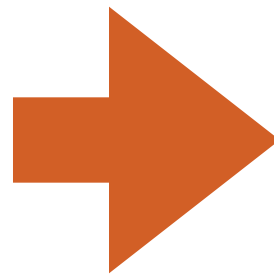


Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution

What if we can use more registers...

```
① movq    (%rdi,%rax), %rsi
② movq    (%rcx,%rax), %r8
③ movq    %r8, (%rdi,%rax)
④ movq    %rsi, (%rcx,%rax)
⑤ addq    $8, %rax
⑥ cmpq    %r9, %rax
⑦ jne     .L9
⑧ movq    (%rdi,%rax), %rsi
⑨ movq    (%rcx,%rax), %r8
⑩ movq    %r8, (%rdi,%rax)
⑪ movq    %rsi, (%rcx,%rax)
⑫ addq    $8, %rax
⑬ cmpq    %r9, %rax
⑭ jne     .L9
```



```
① movq    (%rdi,%rax), %t0
② movq    (%rcx,%rax), %t1
③ movq    %t1, (%rdi,%rax)
④ movq    %t0, (%rcx,%rax)
⑤ addq    $8, %rax, %t2
⑥ cmpq    %r9, %t2
⑦ jne     .L9
⑧ movq    (%rdi, %t2), %t3
⑨ movq    (%rcx, %t2), %t4
⑩ movq    %t4, (%rdi,%t2)
⑪ movq    %t3, (%rcx,%t2)
⑫ addq    $8, %t2, %t5
⑬ cmpq    %r9, %t5
⑭ jne     .L9
```

All false dependencies are gone!!!

The mechanism of OoO: Register renaming + speculative execution

- K. C. Yeager, "The MIPS R10000 superscalar microprocessor," in IEEE Micro, vol. 16, no. 2, pp. 28-41, April 1996.

Register renaming + OoO

- Redirecting the output of an instruction instance to a **physical register**
- Redirecting inputs of an instruction instance from **architectural registers** to correct **physical registers**
 - You need a mapping table between architectural and physical registers
 - You may also need reference counters to reclaim physical registers
- OoO: Executing an instruction all operands are ready (the values of depending physical registers are generated)
 - You will need an **issue logic** to **issue** an instruction to the target functional unit

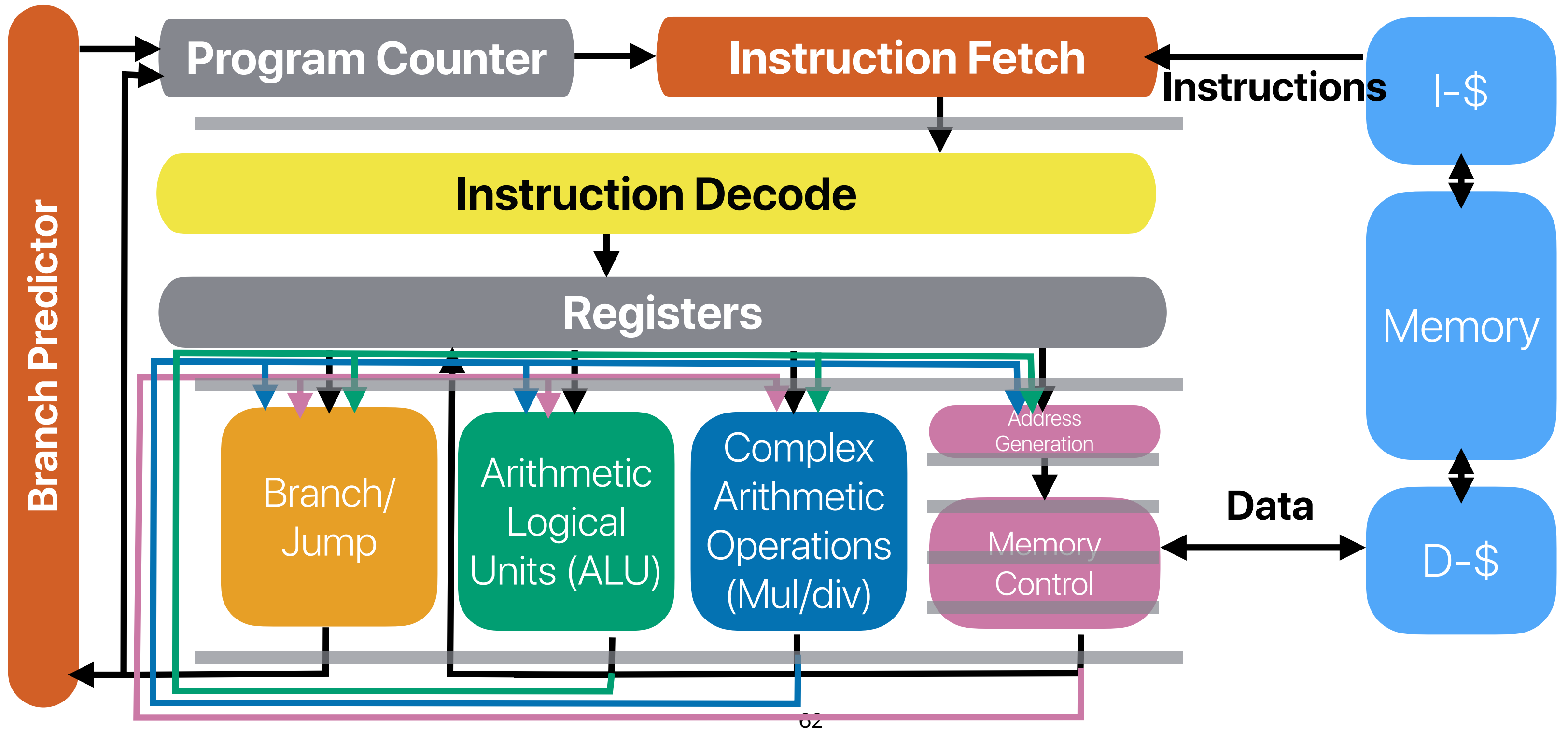
Can we really execute instructions OoO?

- Exceptions may occur anytime — divided by 0, page fault
 - A later instruction cannot write back its own result otherwise the architectural states won't be correct
 - Instructions after the one causes the exception should not be executed
- Hardware can schedule instruction across branch instructions with the help of branch prediction
 - Fetch instructions according to the branch prediction
 - However, branch predictor can never be perfect

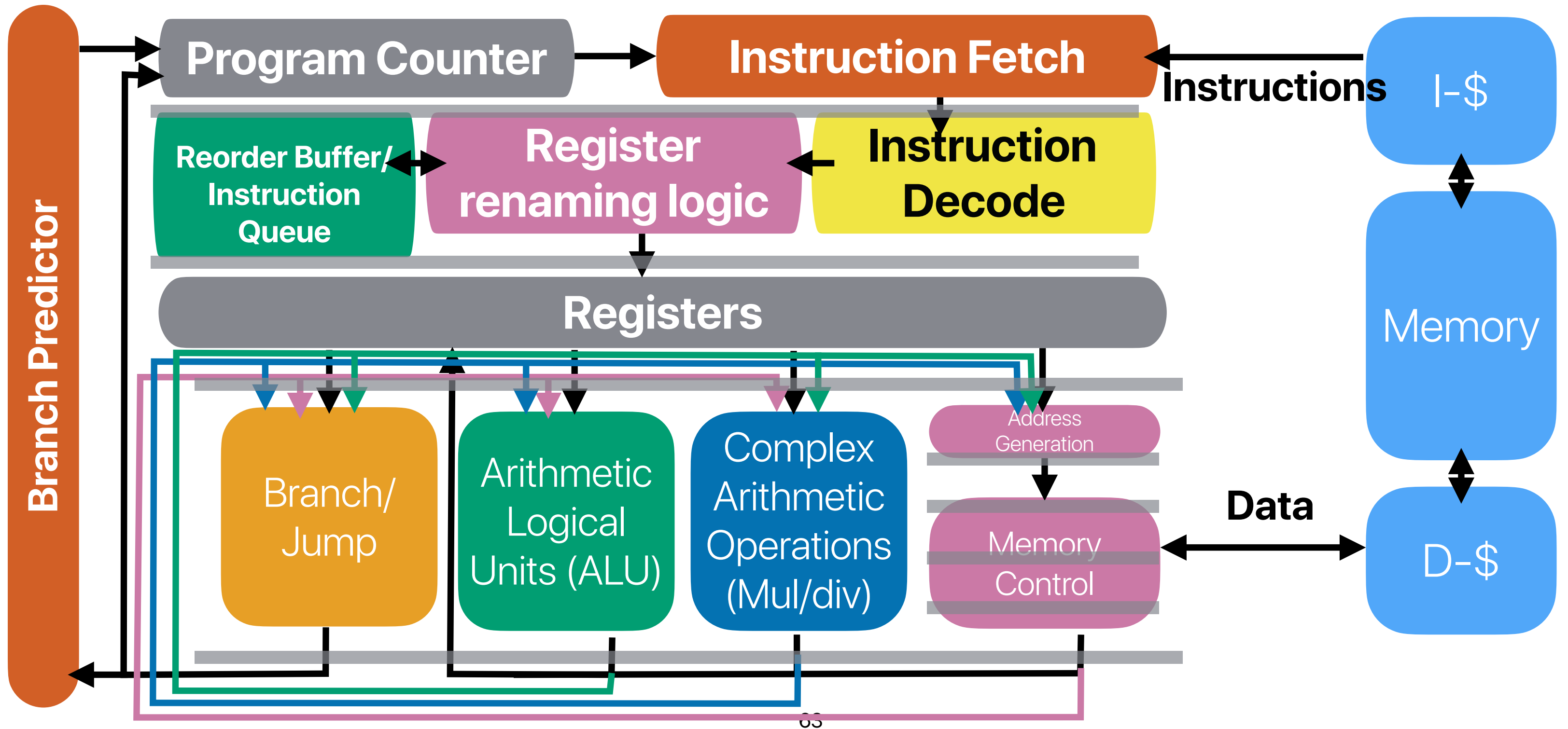
Speculative Execution

- **Speculative** execution mode: an executing instruction is considered as **speculative** before the processor hasn't determined if the instruction should be executed or not
- Reorder buffer (ROB)
 - The processor allocates an entry for each instruction in a reorder buffer
 - Store results in **reorder buffer and physical registers** when the instruction is still speculative
 - If an earlier instruction failed to commit due to an exception or mis-prediction, the physical registers and all ROB entries after the failed-to-commit instruction are flushed
- Commit/Retire
 - Present the execution result to the running program and in architectural registers when all prior instructions are non-speculative
 - Release the ROB entry

Data "forwarding"



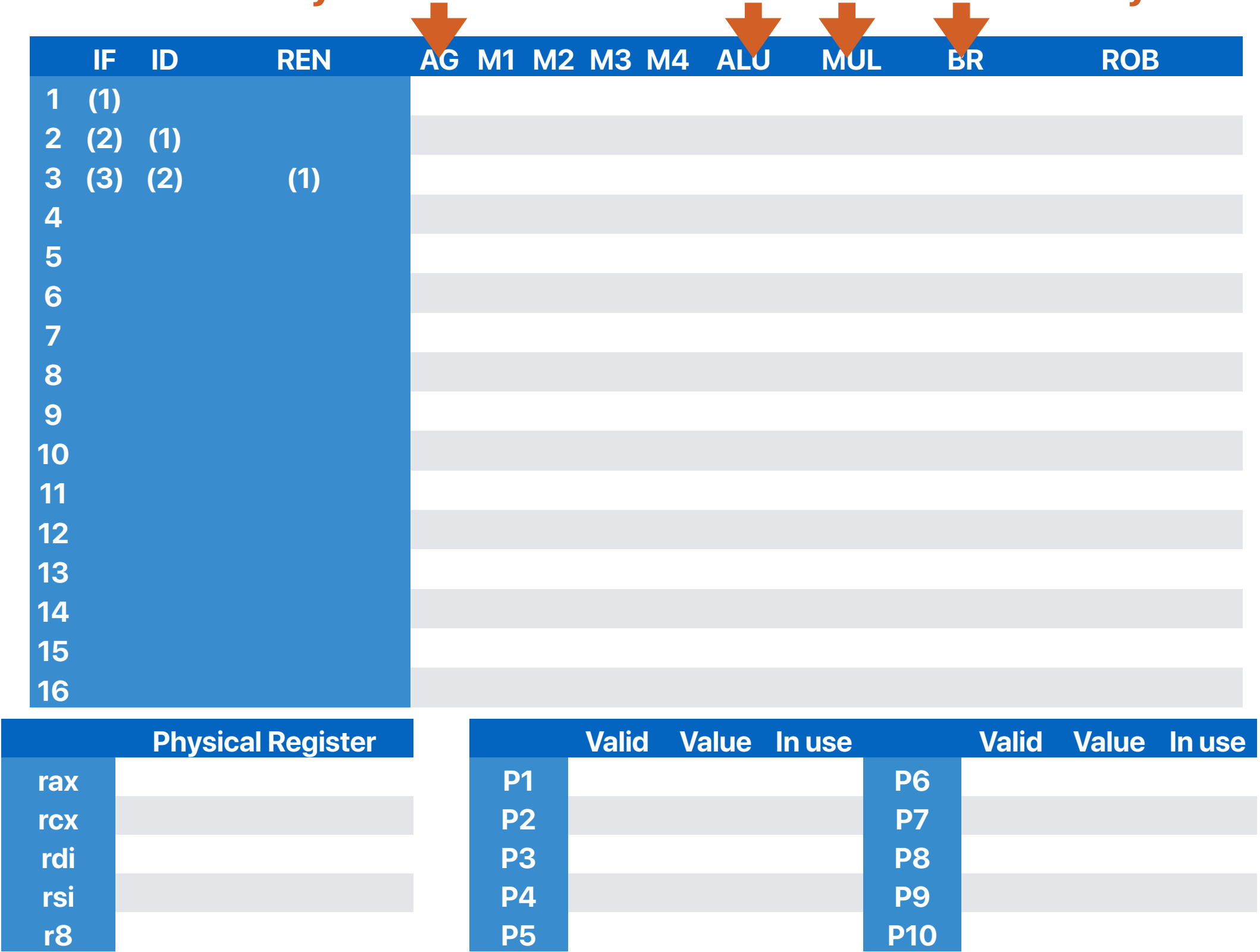
Register renaming + OoO + RoB



Register renaming

Only 1 of them can have a instruction at the same cycle

```
① movq (%rdi,%rax), %rsi
② movq (%rcx,%rax), %r8
③ movq %r8, (%rdi,%rax)
④ movq %rsi, (%rcx,%rax)
⑤ addq $8, %rax
⑥ cmpq %r9, %rax
⑦ jne .L9
⑧ movq (%rdi,%rax), %rsi
⑨ movq (%rcx,%rax), %r8
⑩ movq %r8, (%rdi,%rax)
⑪ movq %rsi, (%rcx,%rax)
⑫ addq $8, %rax
⑬ cmpq %r9, %rax
⑭ jne .L9
```



Register renaming

Only 1 of them can have a instruction at the same cycle

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8
- ③ movq %r8, (%rdi,%rax)
- ④ movq %rsi, (%rcx,%rax)
- ⑤ addq \$8, %rax
- ⑥ cmpq %r9, %rax
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi
- ⑨ movq (%rcx,%rax), %r8
- ⑩ movq %r8, (%rdi,%rax)
- ⑪ movq %rsi, (%rcx,%rax)
- ⑫ addq \$8, %rax
- ⑬ cmpq %r9, %rax
- ⑭ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5												
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

Physical Register			Valid	Value	In use	Valid			Value	In use
rax						P1	0	1	P6	
rcx						P2			P7	
rdi						P3			P8	
rsi		P1				P4			P9	
r8						P5			P10	

Register renaming

Only 1 of them can have a instruction at the same cycle

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8 → P2
- ③ movq %r8, (%rdi,%rax)
- ④ movq %rsi, (%rcx,%rax)
- ⑤ addq \$8, %rax
- ⑥ cmpq %r9, %rax
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi
- ⑨ movq (%rcx,%rax), %r8
- ⑩ movq %r8, (%rdi,%rax)
- ⑪ movq %rsi, (%rcx,%rax)
- ⑫ addq \$8, %rax
- ⑬ cmpq %r9, %rax
- ⑭ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6												
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

Physical Register			Valid	Value	In use				Valid	Value	In use
rax						P1	0		1	P6	
rcx						P2	0		1	P7	
rdi						P3				P8	
rsi		P1				P4				P9	
r8		P2				P5				P10	

Register renaming

Only 1 of them can have a instruction at the same cycle

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8 → P2
- ③ movq %r8, (%rdi,%rax)
- ④ movq %rsi, (%rcx,%rax)
- ⑤ addq \$8, %rax
- ⑥ cmpq %r9, %rax
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi
- ⑨ movq (%rcx,%rax), %r8
- ⑩ movq %r8, (%rdi,%rax)
- ⑪ movq %rsi, (%rcx,%rax)
- ⑫ addq \$8, %rax
- ⑬ cmpq %r9, %rax
- ⑭ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7												
8												
9												
10												
11												
12												
13												
14												
15												
16												

Physical Register	
rax	
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3				P8			
P4				P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi
- ⑨

movq (%rcx,%rax), %r8
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8												
9												
10												
11												
12												
13												
14												
15												
16												

Physical Register			Valid	Value	In use				Valid	Value	In use
rax						P1	0		1	P6	
rcx						P2	0		1	P7	
rdi						P3				P8	
rsi		P1				P4				P9	
r8		P2				P5				P10	

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi
- ⑨

movq (%rcx,%rax), %r8
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8												
9												
10												
11												
12												
13												
14												
15												
16												

Physical Register	
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3	0		1	P8			
P4				P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi
- ⑨

movq (%rcx,%rax), %r8
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)	(2)	(1)							
7	(7)	(6)	(3)(4)(5)		(2)	(1)						
8	(8)	(7)	(3)(4)(6)			(2)	(1)	(5)				
9												
10												
11												
12												
13												
14												
15												
16												

Instruction (5) is running ahead of (3)

Physical Register	
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	0		1	P6			
P2	0		1	P7			
P3	0		1	P8			
P4				P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ jne .L9

```

⑧ movq (%rdi,%rax), %rsi

```

```

⑨ movq (%rcx,%rax), %r8

```

```

movq %rax,%rax,%rax
⑩ movq %r8, (%rdi,%rax)

```

```

⑪ movq %rsi, (%rcx,%rax)

```

```

11  movq %rax, %rax
12  addq $8, %rax

```

```

(13) cmpq %r9, %rax

```

⑭ ine 19

⑭ 2016年12月

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10												
11												
12												
13												
14												
15												
16												

Instruction (4) is running ahead of (3)

Instruction (5) is running ahead of (3)

Physical Register	
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	0		1	P7			
P3	1		1	P8			
P4				P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③  movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ jne .L9

```
⑧ movq (%rdi,%rax), %rsi
```

```

⑨ movq (%rcx,%rax), %r8

```

```

movq %rax,%rax,%rax
⑩ movq %r8, (%rdi,%rax)

```

```

⑪ movq %rsi, (%rcx,%rax)

```

```

11  movq %rax, %rax
12  addq $8, %rax

```

```

(13) cmpq %r9, %rax

```

⑭ ine 19

⑭ 2016年12月

[illegible]

	Physical Register
rax	P3
rcx	
rdi	
rsi	P1
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	0		1	P7			
P3	1		1	P8			
P4				P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11												
12												
13												
14												
15												
16												

Physical Register	
rax	P3
rcx	
rdi	
rsi	P4
r8	P2

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5				P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8 → P2
- ③ movq %r8, (%rdi,%rax)
- ④ movq %rsi, (%rcx,%rax)
- ⑤ addq \$8, %rax → P3
- ⑥ cmpq %r9, %rax
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi → P4
- ⑨ movq (%rcx,%rax), %r8 → P5
- ⑩ movq %r8, (%rdi,%rax)
- ⑪ movq %rsi, (%rcx,%rax)
- ⑫ addq \$8, %rax
- ⑬ cmpq %r9, %rax
- ⑭ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12												
13												
14												
15												
16												

Physical Register	
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8

→ P5
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13												
14												
15												
16												

Physical Register	
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8

→ P5
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14												
15												
16												

Physical Register	
rax	P3
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6			
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8

→ P5
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax

→ P6
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15												
16												

Physical Register	
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	0		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8

→ P5
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax

→ P6
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			(3)(4)(5)(6)(7)
16												

Physical Register	
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	0		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

- ①

movq (%rdi,%rax), %rsi

→ P1
- ②

movq (%rcx,%rax), %r8

→ P2
- ③

movq %r8, (%rdi,%rax)
- ④

movq %rsi, (%rcx,%rax)
- ⑤

addq \$8, %rax

→ P3
- ⑥

cmpq %r9, %rax
- ⑦

jne .L9
- ⑧

movq (%rdi,%rax), %rsi

→ P4
- ⑨

movq (%rcx,%rax), %r8

→ P5
- ⑩

movq %r8, (%rdi,%rax)
- ⑪

movq %rsi, (%rcx,%rax)
- ⑫

addq \$8, %rax

→ P6
- ⑬

cmpq %r9, %rax
- ⑭

jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			(3)(4)(5)(6)(7)
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)

Physical Register	
rax	P6
rcx	
rdi	
rsi	P4
r8	P5

	Valid	Value	In use		Valid	Value	In use
P1	1		1	P6	1		1
P2	1		1	P7			
P3	1		1	P8			
P4	0		1	P9			
P5	0		1	P10			

Register renaming

Only 1 of them can have a instruction at the same cycle

```

①  movq (%rdi,%rax), %rsi → P1
②  movq (%rcx,%rax), %r8  → P2
③  movq %r8, (%rdi,%rax)
④  movq %rsi, (%rcx,%rax)
⑤  addq $8, %rax          → P3
⑥  cmpq %r9, %rax
⑦  jne .L9
⑧  movq (%rdi,%rax), %rsi → P4
⑨  movq (%rcx,%rax), %r8  → P5
⑩  movq %r8, (%rdi,%rax)
⑪  movq %rsi, (%rcx,%rax)
⑫  addq $8, %rax          → P6
⑬  cmpq %r9, %rax
⑭  jne .L9
⑮  movq (%rdi,%rax), %rsi
⑯  movq (%rcx,%rax), %r8
⑰  movq %r8, (%rdi,%rax)
⑱  movq %rsi, (%rcx,%rax)
⑲  addq $8, %rax
⑳  cmpq %r9, %rax
㉑  jne .L9

```

[illegible]

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ ine .L9

⑧ `movq (%rdi,%rax), %rsi` → P4

⑨ `movq (%rcx,%rax), %r8` → P5

```

⑩ movq %r8, (%rdi,%rax)

```

```

10 movq %rsi, (%rax,%rcx,4)
11 movq %rsi, (%rcx,%rax)

```

```

(12) addq $8, %rax

```

```

(13) cmpq %r9, %rax

```

⑪ `inc` `19`

```

14  jne     .L7
15  movq   (%rdi,%rax),%rci

```

```

15) movq    (%101,%1ax), %1s1
16) movq    (%rax,%rax), %r0

```

```

16  movq  (%rcx,%rax), %lo
17  movq  %r8, (%rdi,%rax)

```

```

17  movq  %r8,  (%rdi,%rax)
18  movq  %rci,  (%rax,%rax)

```

```

⑮ movq %rsi, (%rcx,%rax)
⑯ addq $8, %rax

```

```

19 addq $8, %rax
20 cmpq %rax, %rax

```

```

(20) cmpq %r9, %rax
(21) jle 40

```

(21) jne .L9

[illegible]

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ ine .L9

⑧ `movq (%rdi,%rax), %rsi` → P4

⑨ `movq (%rcx,%rax), %r8` → P5

```

(10) movq %r8, (%rdi,%rax)

```

```

10 movq %rsi, (%rax,%rcx,4)
11 movq %rsi, (%rcx,%rax)

```

```

(12) addq $8, %rax → P6

```

```

(13) cmpq %r9, %rax

```

⑪ `inc` `l9`

```

14  jne     .L7
15  movq   (%rdi,%rax),%rci

```

```

15  movq  (%101,%1ax), %1s1
16  movq  (%rax,%rax), %r0

```

```

16  movq  (%rcx,%rax), %lo
17  movq  %r8, (%rdi,%rax)

```

```

17  movq  %r8,  (%r10,%rax)
18  movq  %rcx,  (%rax,%rax)

```

```

⑧ movq %rsi, (%rcx,%rax)
⑩ addq $8, %rax

```

```

19    addq $8, %rax
20    cmpq %rax, %rcx

```

```

(20) cmpq %r9, %rax
(21) jle 10

```

(21) jne .L9

[illegible]

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ ine .L9

⑧ `movq (%rdi,%rax), %rsi` → P4

⑨ `movq (%rcx,%rax), %r8` → P5

```

⑩ movq %r8, (%rdi,%rax)

```

```

10  movq %rsi, (%rax,%rcx,4)
11  movq %rsi, (%rcx,%rax)

```

```

(12) addq $8, %rax → P6

```

```

(13) cmpq %r9, %rax

```

⑭ ine 19

```

14  jne     .L7
15  movq   (%rdi,%rax),%rci

```

```

15  movq    (%rdi,%rax), %rsi
16  movq    (%rax,%rax), %r0

```

```

16  movq  (%rcx,%rax), %lo
17  movq  %r8, (%rdi,%rax)

```

```

17  movq  %r8,  (%r10,%rax)
18  movq  %rcx,  (%rax,%rax)

```

```

⑧ movq %rsi, (%rcx,%rax)
⑩ addq $8, %rax

```

```

19    addq $8, %rax
20    cmpq %rax, %rax

```

```

(20) cmpq %r9, %rax
(21) jle 10

```

(21) jne .L9

[illegible]

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ ine .L9

⑧ `movq (%rdi,%rax), %rsi` → P4

⑨ `movq (%rcx,%rax), %r8` → P5

```

(10) movq %r8, (%rdi,%rax)

```

```

10  movq %rsi, (%rax,%rcx,8)
11  movq %rsi, (%rcx,%rax)

```

```

(12) addq $8, %rax

```

```

(13) cmpq %r9, %rax

```

⑭ ine 19

```

14  jne     .L7
15  movq   (%rdi,%rax),%rci

```

```

15) movq    (%101,%1ax), %1s1
16) movq    (%rax,%rax), %r0

```

```

16  movq  (%rcx,%rax), %lo
17  movq  %r8, (%rdi,%rax)

```

```

17  movq  %r8,  (%rdi,%rax)
18  movq  %rci,  (%rcx,%rax)

```

```

⑩ movq %rsi, (%rcx,%rax)
⑪ addq $8, %rax

```

```

19 addq $8, %rax
20 cmpq %rax, %rcx

```

```

(20) cmpq %r9, %rax
(21) jle 40

```

(21) jne .L9

[illegible]

Register renaming

Only 1 of them can have a instruction at the same cycle

① `movq (%rdi,%rax), %rsi` → P1

② `movq (%rcx,%rax), %r8` → P2

```
③ movq %r8, (%rdi,%rax)
```

```
④ movq %rsi, (%rcx,%rax)
```

⑤ `addq $8, %rax` → P3

```
⑥    cmpq %r9, %rax
```

⑦ ine .L9

⑧ `movq (%rdi,%rax), %rsi` → P4

⑨ `movq (%rcx,%rax), %r8` → P5

```

(10) movq %r8, (%rdi,%rax)

```

```

10 movq %rsi, (%rax,%rcx,8)
11 movq %rsi, (%rcx,%rax)

```

```

(12) addq $8, %rax

```

```

(13) cmpq %r9, %rax

```

⑭ ine 19

```

14  jne     .L7
15  movq   (%rdi,%rax),%rci

```

```

15) movq    (%101,%1ax), %1s1
16) movq    (%rax,%rax), %r0

```

```

16  movq  (%rcx,%rax), %lo
17  movq  %r8, (%rdi,%rax)

```

```

17  movq  %r8,  (%rdi,%rax)
18  movq  %rci,  (%rcx,%rax)

```

```

⑮ movq %rsi, (%rcx,%rax)
⑯ addq $8, %rax

```

```

19 addq $8, %rax
20 cmpq %r0, %rax

```

```

(20) cmpq %r9, %rax
(21) jle 10

```

(21) jne .L9

[illegible]

Only 1 of them can have a instruction at the same cycle

	IF	ID	REN	AG	M1	M2	M3	M4
1	(1)							

①	movq (%rdi,%rax), %rsi	→ P1
②	movq (%rcx,%rax), %r8	→ P2
③	movq %r8, (%rdi,%rax)	
④	movq %rsi, (%rcx,%rax)	
⑤	addq \$8, %rax	→ P3
⑥	cmpq %r9, %rax	
⑦	jne .L9	
⑧	movq (%rdi,%rax), %rsi	→ P4
⑨	movq (%rcx,%rax), %r8	→ P5
⑩	movq %r8, (%rdi,%rax)	
⑪	movq %rsi, (%rcx,%rax)	
⑫	addq \$8, %rax	→ P6
⑬	cmpq %r9, %rax	
⑭	jne .L9	
⑮	movq (%rdi,%rax), %rsi	
⑯	movq (%rcx,%rax), %r8	
⑰	movq %r8, (%rdi,%rax)	
⑱	movq %rsi, (%rcx,%rax)	
⑲	addq \$8, %rax	
⑳	cmpq %r9, %rax	
\textcircled{21}	jne .L9	

→ P1

→ P2

→ P3

→ P4

→ P5

→ P6

Only 1 of them can have a instruction at the same cycle

Register renaming

- ① movq (%rdi,%rax), %rsi → P1
- ② movq (%rcx,%rax), %r8 → P2
- ③ movq %r8, (%rdi,%rax)
- ④ movq %rsi, (%rcx,%rax)
- ⑤ addq \$8, %rax → P3
- ⑥ cmpq %r9, %rax
- ⑦ jne .L9
- ⑧ movq (%rdi,%rax), %rsi → P4
- ⑨ movq (%rcx,%rax), %r8 → P5
- ⑩ movq %r8, (%rdi,%rax)
- ⑪ movq %rsi, (%rcx,%rax)
- ⑫ addq \$8, %rax → P6
- ⑬ cmpq %r9, %rax
- ⑭ jne .L9
- ⑮ movq (%rdi,%rax), %rsi
- ⑯ movq (%rcx,%rax), %r8
- ⑰ movq %r8, (%rdi,%rax)
- ⑱ movq %rsi, (%rcx,%rax)
- ⑲ addq \$8, %rax
- ⑳ cmpq %r9, %rax
- ㉑ jne .L9

	IF	ID	REN	AG	M1	M2	M3	M4	ALU	MUL	BR	ROB
1	(1)											
2	(2)	(1)										
3	(3)	(2)	(1)									
4	(4)	(3)	(2)	(1)								
5	(5)	(4)	(3)	(2)	(1)							
6	(6)	(5)	(3)(4)		(2)	(1)						
7	(7)	(6)	(3)(4)(5)			(2)	(1)					
8	(8)	(7)	(3)(4)(6)				(2)	(1)	(5)			
9	(9)	(8)	(3)(6)(7)	(4)				(2)				(1)(5)
10	(10)	(9)	(6)(7)(8)	(3)	(4)							(2)(5)
11	(11)	(10)	(7)(8)(9)		(3)	(4)			(6)			
12	(12)	(11)	(8)(9)(10)			(3)	(4)				(7)	(5)(6)
13	(13)	(12)	(9)(10)(11)	(8)			(3)	(4)				(5)(6)(7)
14	(14)	(13)	(10)(11)(12)	(9)	(8)			(3)				(4)(5)(6)(7)
15	(15)	(14)	(10)(11)(13)		(9)	(8)			(12)			(3)(4)(5)(6)(7)
16	(16)	(15)	(10)(11)(14)			(9)	(8)		(13)			(12)
17	(17)	(16)	(10)(11)(15)				(9)	(8)			(14)	(12)(13)
18	(18)	(17)	(10)(15)(16)	(11)				(9)				(8)(12)(13)(14)
19	(19)	(18)	(15)(16)(17)	(10)	(11)							(9)(12)(13)(14)
20	(20)	(19)	(16)(17)(18)	(15)	(10)	(11)						(12)(13)(14)
21	(21)	(20)	(17)(18)(19)	(16)	(15)	(10)	(11)					(12)(13)(14)
22		(21)	(17)(18)(20)		(16)	(15)	(10)	(11)	(19)			(12)(13)(14)
23			(17)(20)(21)	(18)		(16)	(15)	(10)				(11)(12)(13)(14)(19)
24			(20)(21)	(17)	(18)		(16)	(15)				(10)(11)(12)(13)(14)(19)
25			(21)		(17)	(18)		(16)	(20)			(15)(19)

7 cycles for 7 instructions
CPI = 1

Takeaways: data hazards

- More data dependencies, more likelihood of data hazards
- Stalls and data forwarding can both address data hazards to generate correct code execution results — but not very efficient
- Compiler optimizations can help, but to a limited extent
- False dependencies limits the freedom of out-of-order execution
- Register renaming + Speculative execution enables more efficient execution by dynamically scheduling instructions whenever their data dependencies are resolved

Announcements

- **Assignment 3 & Programming Assignment 2 for CSE142L due this Sunday**
- **Reading Quiz 7 due next Wednesday** before the lecture
- **No lecture next Monday**

Computer Science & Engineering

142

つづく

